Regimens API Schema

COE 454 Software Engineering | Team Regimens

Outstretched Arm Enterprise | REST + JWT | v1.1

This document is the contract between the React client and the Express API. It is generated from the implemented routes (backend/src/routes). Machine-readable copy: docs/API_Schema_Regimens.yaml (OpenAPI 3.0.3).

Base URL local: http://localhost:4000 Production: https://<service>.onrender.com

Content-Type: application/json. Auth: Authorization: Bearer <token>. Errors: { "message": "..." }.

1. Conventions

Public: POST /api/auth/login and GET /api/health. All other routes require a valid JWT (401 if missing/expired).

Owner-only (403 otherwise): GET/POST /api/auth/users, all /api/backup* .

Roles: owner | assistant. Demo login: owner / owner1234 (change after deploy).

JSON numbers for money are NUMERIC(10,2) in Postgres. The client never sends sale prices; the server stamps them.

USE_MOCK_DB=true uses in-memory data (health.mode=mock). false uses Postgres (health.mode=postgres). Same paths.

CORS: localhost always allowed; production CORS_ORIGIN should include *.vercel.app. Body limit 256kb. Login rate-limited.

Common codes: 200 OK, 201 Created, 400 bad backup path, 401 unauthorized, 403 forbidden/origin, 404 not found, 409 conflict (SKU/username/stock), 422 validation, 500 server.

2. Endpoint catalogue

Health

GET **/api/health**

[public] Returns { status: ok, mode: mock|postgres }

Auth

POST **/api/auth/login**

[public] Body { username, password } -> { token, user }

POST **/api/auth/logout**

[JWT] Acknowledges logout (stateless JWT)

GET **/api/auth/me**

[JWT] Current user { id, name, username, role }

PATCH **/api/auth/profile**

[JWT] Body { name?, username? } -> new token

POST **/api/auth/change-password**

[JWT] Body { currentPassword, newPassword } min 8 chars

GET **/api/auth/users**

[owner] List staff

POST **/api/auth/users**

[owner] Create user { name, username, password, role? }

Products

GET **/api/products**

[JWT] Query search, lowStock=true. Each product includes stock_layers[]

GET **/api/products/:id**

[JWT] One product + remaining FIFO layers

POST **/api/products**

[JWT] Create. name required. Opening qty becomes first layer

PUT **/api/products/:id**

[JWT] Update listed fields. Qty increase = new cost layer

DELETE **/api/products/:id**

[JWT] { message: Product deleted }

Categories

GET **/api/categories**
[JWT] List

GET **/api/categories/:id**
[JWT] One

POST **/api/categories**
[JWT] { name, description?, keywords? }

PUT **/api/categories/:id**
[JWT] Partial update

DELETE **/api/categories/:id**
[JWT] Delete

Sales

GET **/api/sales**
[JWT] Query from, to. Nested items[]

GET **/api/sales/:id**
[JWT] One sale

POST **/api/sales**
[JWT] { items:[{product_id,quantity}], client_created_at? } 201

Reports

GET **/api/reports/sales**
[JWT] period=daily|weekly|monthly|yearly { totalSales, totalRevenue, breakdown }

GET **/api/reports/profit**
[JWT] period required-valid. { revenue, cogs, grossProfit, marginPercent, breakdown }

Backup (owner)

GET **/api/backup**
[owner] List records

POST **/api/backup**
[owner] Create snapshot v3 (users, products, categories, sales, sale_items, stock_layers)

POST **/api/backup/:id/restore**
[owner] Destructive restore of a completed backup

Sync

POST **/api/sync**
[JWT] { sales[], products[] } -> { synced[], conflicts[] } last-write-wins on products

3. Resource schemas

Product

id, name, sku, quantity, unit_price (current shelf price), min_stock_level, category, cost_price (next restock default), created_at, updated_at, stock_layers[].

stock_layers[]: id, product_id, unit_cost, unit_price (listed price when the lot arrived), quantity_remaining, created_at. Only remaining>0 lots are returned. Oldest created_at is sold first.

Sale / sale item

Sale: id, user_id, total_amount, sync_status (synced|pending), created_at, items[].

Item: product_id, quantity, unit_price (stamped shelf price), unit_cost (FIFO-weighted cost), subtotal = unit_price * quantity. Client MUST NOT send prices on POST /sales.

Category

id, name, description, keywords, created_at, updated_at.

Profit report

revenue, cogs, grossProfit (= revenue - cogs), marginPercent, lines, missingCostLines, salesCount, breakdown[{ label, revenue, cogs, profit, salesCount }].

Auth user

Public user never includes password_hash. JWT payload: id, name, role, username. Default expiry 12h (JWT_EXPIRES_IN).

4. Behaviour notes (for frontend and markers)

POST /sales in connected mode locks the product row (SELECT FOR UPDATE), checks quantity, consumes FIFO layers, decrements products.quantity, inserts sale + items. 409 if not enough stock. 404 if product missing.

Raising unit_price does not rewrite old layers' costs. Raising quantity with a new cost_price opens a new layer. Completed sales keep stamped unit_cost forever.

POST /sync replays queued sales through the same createSale path (max 100 sales, 100 products). Product updates are field-whitelisted. If client_updated_at is older than server updated_at, the edit is a conflict (discarded).

Backup restore truncates operational tables and reloads snapshot version 3. Owner-only. Paths are constrained under database/backups/.

GET /api/reports/sales buckets total_amount. GET /api/reports/profit uses stamped line unit_price and unit_cost, so analytics stay correct after price changes.

5. Example calls

```
POST /api/auth/login
{"username":"owner","password":"owner1234"}

POST /api/sales
{"items":[{"product_id":1,"quantity":2}]}

PUT /api/products/1   (restock + new cost)
{"quantity":55,"cost_price":4,"unit_price":6}

GET /api/reports/profit?period=monthly
```

End of schema. OpenAPI file for Swagger/Postman: API_Schema_Regimens.yaml